

Nama : Imron
NPM : 232310021
Mata Kuliah : Lab PPB TI24PA2
Praktikum 7
1. Latihan 1

Halaman manajemen profil dirancang untuk memfasilitasi pengguna dalam memperbarui informasi personal secara dinamis menggunakan state management bawaan React Native. Alur sistem dimulai ketika komponen Profile pertama kali dimuat (mount) dengan menampilkan data default pada kolom input nama, email, bio, serta gambar avatar yang bersumber dari URI penampung. Ketika pengguna berinteraksi untuk mengubah foto profil melalui tombol yang disediakan, sistem akan memicu sebuah dialog pilihan berbasis Alert untuk menentukan metode pengambilan gambar, yaitu melalui fitur kamera langsung atau memilih berkas yang sudah ada di galeri perangkat.

Proses modifikasi media ini mengintegrasikan library expo-image-picker. Sebelum modul kamera atau galeri dieksekusi, sistem melakukan pemeriksaan hak akses terlebih dahulu menggunakan fungsi asinkron requestCameraPermissionsAsync atau requestMediaLibraryPermissionsAsync. Jika izin ditolak oleh pengguna, aplikasi akan membatalkan proses dan menampilkan pesan peringatan penolakan akses. Sebaliknya, jika izin diberikan, perangkat keras yang dipilih akan aktif dan menyediakan antarmuka pemotongan gambar dengan rasio tetap 1:1 serta optimasi kualitas gambar sebesar 70%. Hasil modifikasi tersebut mengembalikan sebuah URI lokal baru yang kemudian disimpan ke dalam state avatar untuk langsung memperbarui tampilan visual foto profil pada layar secara real-time.

Pada bagian penginputan data teks, antarmuka dibungkus menggunakan komponen KeyboardAvoidingView guna memastikan komponen input tidak tertutup oleh keyboard virtual saat pengguna melakukan pengetikan, baik pada platform Android maupun iOS. Setelah pengguna selesai memanipulasi data pada komponen TextInput, alur akhir diselesaikan melalui fungsi handleSaveChanges. Fungsi ini mengompilasi seluruh variabel status terbaru termasuk data teks dan URI gambar profil ke dalam satu objek data terpadu (updatedData) untuk siap dikirimkan ke log konsol sebagai simulasi pengiriman data menuju database atau Application Programming Interface (API) eksternal, kemudian diakhiri dengan konfirmasi sukses kepada pengguna.

```

profile.js

import * as ImagePicker from "expo-image-picker";
import { useState } from "react";
import {
  Alert,
  Image,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StyleSheet,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from "react-native";

const Profile = () => {
  const [name, setName] = useState("Irnon");
  const [email, setEmail] = useState("irnon@example.com");
  const [bio, setBio] = useState(
    "Information Systems Student | Web & Mobile Developer",
  );

  const [avatar, setAvatar] = useState("https://via.placeholder.com/150");

  const pickerOptions = {
    mediaTypes: ["images"],
    allowsEditing: true,
    aspect: [1, 1],
    quality: 0.7,
  };

  const handleTakeAction = async () => {
    const permissionResult = await ImagePicker.requestCameraPermissionsAsync();

    if (permissionResult.granted === false) {
      Alert.alert(
        "Irin Ditolak",
        "Aplikasi memerlukan akses kamera untuk mengambil foto profil.",
      );
      return;
    }

    const result = await ImagePicker.launchCameraAsync(pickerOptions);

    if (!result.canceled && result.assets[0].uri) {
      setAvatar(result.assets[0].uri);
    }
  };

  const handlePickImage = async () => {
    const permissionResult =
      await ImagePicker.requestMediaLibraryPermissionsAsync();

    if (permissionResult.granted === false) {
      Alert.alert(
        "Irin Ditolak",
        "Aplikasi memerlukan akses galeri untuk memilih foto profil.",
      );
      return;
    }

    const result = await ImagePicker.launchImageLibraryAsync(pickerOptions);

    if (!result.canceled && result.assets[0].uri) {
      setAvatar(result.assets[0].uri);
    }
  };

  const requestAvatarChange = () => {
    Alert.alert(
      "Ubah Avatar",
      "Pilih metode untuk mengubah foto profil Anda:",
      [
        { text: "Kamera", onPress: handleTakeAction },
        { text: "Galeri", onPress: handlePickImage },
        { text: "Batal", style: "cancel" },
      ],
      { cancelable: true },
    );
  };

  const handleSaveChanges = () => {
    const updatedData = { name, email, bio, avatar };
    console.log("Simpan ke Database/API:", updatedData);
    Alert.alert("Sukses", "Profil berhasil diperbaharui!");
  };

  return (
    <KeyboardAvoidingView
      style={styles.container}
      behavior={Platform.OS === "ios" ? "padding" : "height"}
    >
      <ScrollView contentContainerStyle={styles.scrollContainer}>
        <View style={styles.avatarContainer}>
          <Image source={{ uri: avatar }} style={styles.avatar} />
          <TouchableOpacity
            style={styles.changeAvatarBtn}
            onPress={requestAvatarChange}>
            <Text style={styles.changeAvatarText}>Ubah Foto Profil</Text>
          </TouchableOpacity>
        </View>

        <View style={styles.formContainer}>
          <Text style={styles.label}>Nama Lengkap</Text>
          <TextInput
            style={styles.input}
            value={name}
            onChangeText={setName}
            placeholder="Nama Anda"
          />

          <Text style={styles.label}>Email</Text>
          <TextInput
            style={styles.input}
            value={email}
            onChangeText={setEmail}
            placeholder="Email Anda"
            keyboardType="email-address"
            autoCapitalize="none"
          />

          <Text style={styles.label}>Bio</Text>
          <TextInput
            style={[[styles.input, styles.textArea]]}
            value={bio}
            onChangeText={setBio}
            placeholder="Bio singkat"
            multiline
            numberOfLines={3}
          />
        </View>

        <TouchableOpacity style={styles.saveButton} onPress={handleSaveChanges}>
          <Text style={styles.saveButtonText}>Simpan Perubahan</Text>
        </TouchableOpacity>
      </ScrollView>
    </KeyboardAvoidingView>
  );
};

export default Profile;

```

2. Latihan 2

Fitur pemindaian buku berbasis kode QR diimplementasikan memanfaatkan fungsionalitas modul kamera native melalui library expo-camera. Logika program diinisialisasi di dalam hook `useEffect` yang berjalan secara otomatis saat layar Scan dibuka. Pada tahap awal ini, sistem menjalankan fungsi asinkron untuk meminta hak akses penggunaan kamera belakang perangkat keras melalui `Camera.requestCameraPermissionsAsync()`. Status respons dari perangkat akan disimpan ke dalam state `hasPermission`. Selama proses pengecekan hak akses berlangsung, sistem mengembalikan status pemuatan berupa teks informasi kepada pengguna. Apabila hasil akhir dari hak akses bernilai salah atau ditolak, antarmuka akan beralih menampilkan pesan kesalahan yang menginstruksikan pengguna untuk mengaktifkan izin secara manual melalui pengaturan sistem operasi perangkat.

Jika hak akses terkonfirmasi bernilai benar, aplikasi akan langsung merender komponen `CameraView` yang dikonfigurasi secara spesifik untuk hanya merespons jenis kode batang berupa QR Code melalui properti `barcodeTypes: ["qr"]`. Secara visual, antarmuka dilengkapi dengan lapisan atas (overlay container) dan penanda batas pemindaian (scanner marker) berbentuk kotak di bagian tengah layar untuk mempermudah pengguna memposisikan kode QR secara presisi. Kamera akan terus berada dalam kondisi siap memindai hingga mendeteksi adanya masukan data kode QR yang valid melalui event handler `onBarcodeScanned`.

Begitu kode QR berhasil dibaca oleh sensor kamera, fungsi `handleBarcodeScanned` akan langsung dipicu dan mengubah state `scanned` menjadi benar untuk mengunci sementara aktivitas pemindaian agar tidak terjadi pembacaan ganda yang berulang. Sistem kemudian memvalidasi data string yang diekstrak dari kode QR tersebut; jika data tersedia, komponen router dari expo-router akan melakukan navigasi dinamis dengan mengarahkan pengguna menuju halaman detail buku spesifik menggunakan parameter ID buku yang didapat dari hasil pemindaian (`/books/${data}`). Jika data kosong atau tidak dikenali, sistem memunculkan dialog peringatan kegagalan. Guna memulihkan fungsionalitas pemindai untuk penggunaan berikutnya, sebuah fungsi `setTimeout` dipasang untuk mengembalikan status state `scanned` menjadi salah setelah jeda waktu dua detik.

```

scan.jsx

import { Camera, CameraView } from "expo-camera";
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import { Alert, StyleSheet, Text, View } from "react-native";

export default function ScanScreen() {
  const router = useRouter();
  const [hasPermission, setHasPermission] = useState(null);
  const [scanned, setScanned] = useState(false);

  useEffect(() => {
    const getCameraPermissions = async () => {
      const { status } = await Camera.requestCameraPermissionsAsync();
      setHasPermission(status === "granted");
    };
    getCameraPermissions();
  }, []);

  const handleBarcodeScanned = ({ type, data }) => {
    setScanned(true);

    if (data) {
      router.push(`/books/${data}`);
    } else {
      Alert.alert("Gagal", "QR Code tidak valid.");
    }

    setTimeout(() => {
      setScanned(false);
    }, 2000);
  };

  if (hasPermission === null) {
    return (
      <View style={styles.center}>
        <Text style={styles.textInfo}>Meminta izin akses kamera ... </Text>
      </View>
    );
  }
  if (hasPermission === false) {
    return (
      <View style={styles.center}>
        <Text style={styles.textError}>
          Akses kamera ditolak. Harap izinkan di pengaturan perangkat Anda.
        </Text>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <CameraView
        onBarcodeScanned={scanned ? undefined : handleBarcodeScanned}
        barcodeScannerSettings={{
          barcodeTypes: ["qr"],
        }}
        style={StyleSheet.absoluteFillObject}
      />

      <View style={styles.overlayContainer}>
        <View style={styles.scannerMarker} />
        <Text style={styles.hintText}>
          Posisikan QR Code Buku di dalam kotak
        </Text>
      </View>
    </View>
  );
}

```

3. Latihan 3

Halaman detail buku dirancang untuk menyajikan informasi komprehensif mengenai buku yang dipilih sekaligus menyediakan modul pemutar suara (Text-to-Speech) interaktif. Alur sistem dimulai ketika komponen Detail menerima parameter ID buku secara dinamis melalui hook `useLocalSearchParams()`. Berdasarkan ID tersebut, sistem melakukan pencarian data pada larik objek `ListBook` menggunakan metode `.find()`. Jika data buku tidak ditemukan, sistem akan memotong alur eksekusi dan merender antarmuka fallback berupa pesan kesalahan `Book not found`. Namun, jika data terkonfirmasi ada, antarmuka utama akan memuat komponen `ImageBackground` yang menerapkan efek blur statis dari cover buku sebagai latar belakang estetik, serta menampilkan detail tekstual seperti judul, nama penulis, nilai rating, dan ringkasan sinopsis di dalam wadah penjelajah `ScrollView`.

Logika fitur pemutar audio (audio-book) dikelola secara terpisah dengan memanfaatkan library `expo-speech`. Sebelum pemrosesan suara dilakukan, data teks cerita panjang yang bersumber dari properti `book.story` diekstraksi terlebih dahulu melalui manipulasi string `.split("\n")` guna memecah teks menjadi potongan-potongan paragraf terpisah di dalam array `isiParagrafArray`. Ketika pengguna menekan tombol `Read Now`, sistem akan memunculkan sebuah komponen Modal yang bertindak sebagai panel pemutar audio. Eksekusi pemutaran suara dikendalikan secara asinkron oleh fungsi `playSpeech`, yang mengirimkan potongan teks paragraf aktif ke fungsi bawaan `Speech.speak()`. Fungsi ini secara dinamis menyesuaikan aksentasi atau logat suara robot berdasarkan properti `book.language` bawaan data (misalnya aksentasi `"en-US"` untuk bahasa Inggris dan `"id"` untuk bahasa Indonesia).

Selama audio berjalan, sistem mengimplementasikan penanda posisi bacaan dinamis (text highlighting). Indeks paragraf yang sedang dibacakan oleh mesin speech disimpan ke dalam state `currentParagraphIndex`, yang kemudian memicu perubahan gaya visual (styling) pada baris teks terkait berupa latar belakang warna kuning cerah menyerupai stabilo. Melalui mekanisme callback loop memanfaatkan properti `onDone`, fungsi `playSpeech` akan memanggil dirinya sendiri secara rekursif untuk melanjutkan pembacaan ke indeks paragraf berikutnya secara otomatis hingga seluruh isi cerita selesai dibacakan. Pengguna juga diberikan kontrol penuh untuk menghentikan sementara suara melalui fungsi `pauseSpeech` yang mengeksekusi `Speech.stop()`, maupun menutup panel modal lewat fungsi `handleCloseModal` yang secara otomatis membersihkan (cleanup) seluruh proses audio yang sedang berjalan agar tidak terjadi kebocoran memori pada perangkat.

[illegible]